

Ford–Fulkerson and Edmonds–Karp Algorithms

1 Maximum Flow

A flow network is a directed graph in which each edge has a capacity. It contains a source s , where the flow originates, and a sink t , where the flow terminates.

The objective of the maximum-flow problem is to determine the maximum possible flow from s to t without exceeding the capacity of any edge.

For an edge (u, v) ,

$$0 \leq f(u, v) \leq c(u, v)$$

where $f(u, v)$ is the flow through the edge and $c(u, v)$ is its capacity.

For every vertex other than the source and sink, the total incoming flow must equal the total outgoing flow.

2 Ford–Fulkerson Algorithm

The Ford–Fulkerson algorithm finds a maximum flow by repeatedly finding an **augmenting path** from the source to the sink in the residual graph.

An augmenting path is a path along which additional flow can be sent.

2.1 Residual Capacity

The residual capacity of an edge is the amount of additional flow that can be sent through it:

$$c_f(u, v) = c(u, v) - f(u, v)$$

When flow is sent through an edge, a reverse edge is added to the residual graph. This allows previously assigned flow to be reduced or redirected if necessary.

2.2 Algorithm Steps

1. Initialize the flow of every edge to zero.
2. Construct the residual graph.
3. Find an augmenting path from s to t .
4. Find the minimum residual capacity on the path.

5. Increase the flow along the path by this value.
6. Update the residual graph.
7. Repeat until no augmenting path exists.

When no augmenting path remains, the resulting flow is a maximum flow.

2.3 Pseudocode

```

FordFulkerson(G, s, t):

    Set flow of every edge to 0

    while an augmenting path P exists:
        delta = minimum residual capacity on P

        for each edge (u, v) in P:
            increase flow(u, v) by delta
            update the reverse residual edge

    return maximum flow

```

2.4 Example

Consider the following flow network:

$$s \xrightarrow{10} A \xrightarrow{8} t$$

$$s \xrightarrow{5} B \xrightarrow{10} t$$

First, choose the augmenting path

$$s \rightarrow A \rightarrow t$$

The bottleneck capacity is

$$\min(10, 8) = 8.$$

Therefore, 8 units of flow are sent.

Next, choose

$$s \rightarrow B \rightarrow t.$$

The bottleneck capacity is

$$\min(5, 10) = 5.$$

Therefore, another 5 units of flow are sent.

The total flow is

$$|f| = 8 + 5 = 13.$$

No additional flow can leave the source. Therefore,

Maximum Flow = 13

2.5 Complexity

Let V be the number of vertices, E the number of edges, and F the value of the maximum flow.

For integer capacities, each augmentation increases the flow by at least one unit. There can therefore be at most F augmentations.

Finding an augmenting path takes $O(E)$ time.

Hence, the time complexity is

$O(EF)$

for integer capacities.

3 Edmonds–Karp Algorithm

The Edmonds–Karp algorithm is a specific implementation of the Ford–Fulkerson method.

The main difference is how the augmenting path is selected.

Ford–Fulkerson can use any augmenting path, whereas Edmonds–Karp always selects the shortest augmenting path using **Breadth-First Search (BFS)**.

The shortest path is measured by the number of edges.

3.1 Algorithm Steps

1. Initialize all flows to zero.
2. Construct the residual graph.
3. Use BFS to find the shortest augmenting path from s to t .
4. If no path exists, terminate.
5. Find the bottleneck residual capacity.
6. Increase the flow by the bottleneck value.
7. Update the residual graph.
8. Repeat until no augmenting path exists.

3.2 Pseudocode

```
EdmondsKarp(G, s, t):  
  
    Set flow of every edge to 0  
  
    while BFS finds an augmenting path P:  
        delta = minimum residual capacity on P  
  
        for each edge (u, v) in P:  
            increase flow(u, v) by delta  
            update the reverse residual edge  
  
    return maximum flow
```

3.3 Example

Using the same network,

$$s \xrightarrow{10} A \xrightarrow{8} t$$

$$s \xrightarrow{5} B \xrightarrow{10} t$$

BFS first finds

$$s \rightarrow A \rightarrow t.$$

The bottleneck capacity is

$$\min(10, 8) = 8.$$

Thus, 8 units of flow are sent.

BFS then finds

$$s \rightarrow B \rightarrow t.$$

The bottleneck capacity is

$$\min(5, 10) = 5.$$

Therefore,

$$|f| = 8 + 5 = 13.$$

No further augmenting path exists, so

Maximum Flow = 13

3.4 Complexity

Each BFS takes

$$O(E)$$

time.

The number of augmentations in Edmonds–Karp is bounded by

$$O(VE).$$

Therefore, the overall time complexity is

$$\boxed{O(VE^2)}$$

The space complexity is

$$\boxed{O(V + E)}.$$

4 Comparison

Feature	Ford–Fulkerson	Edmonds–Karp
Path selection	Any augmenting path	Shortest augmenting path
Path-finding method	Any suitable method	BFS
Time complexity	$O(EF)$ for integer capacities	$O(VE^2)$
Space complexity	$O(V + E)$	$O(V + E)$

5 Conclusion

Ford–Fulkerson finds the maximum flow by repeatedly augmenting flow along available paths in the residual graph.

Edmonds–Karp is a specific implementation of Ford–Fulkerson that uses BFS to select the shortest augmenting path. Its time complexity is

$$\boxed{O(VE^2)}.$$